

INFO0902-1 : Projet 2 - Bin packing

Romain Malaise, Baptiste Renard

5 juillet 2026

1 Analyse théorique

1.1 Idée de la complexité approximative du problème par recherche exhaustive

D'abord, nous pouvons voir le problème comme tester toutes les possibilités de répartition de N fichiers dans N disques, c'est à dire N^N possibilités. Ensuite, nous retenons la distribution valable qui minimise le nombre de disques utilisés. La complexité par recherche exhaustive est donc de l'ordre de $O(N^N)$, ce qui est inexploitable en pratique. Cependant, en réalité, nous savons déjà que le nombre de distributions valables est moindre que N^N , parce que beaucoup de distributions ne sont pas valables car elles dépassent la capacité d'un des disques. Enfin, nous pouvons affirmer que $O(N^N)$ est bien une borne maximale sur la complexité du problème.

1.2 Preuve que deux des quatres solutions gloutonnes ne sont pas optimales

Deux des quatre solutions gloutonnes ne sont pas optimales, en effet, nous pouvons trouver des contre-exemples pour deux de ces algorithmes (présentés sur la figure ci-dessous). Nous prenons cinq fichiers de tailles respectives 600;500;400;300;200 et des disques de capacité de 1000. En supposant, que nous commençons par distribuer les fichiers de plus grandes tailles en premier, nous avons que la solution du NextFit est trois disques et celle du WorstFit est aussi trois disques. Cependant, il est évident que la solution optimale est deux disques.

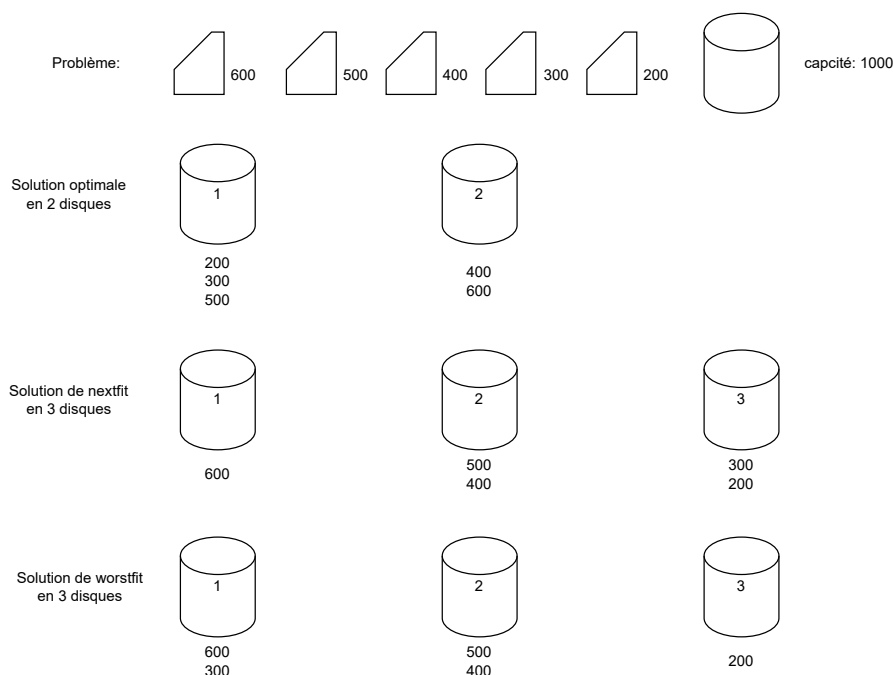


FIGURE 1 – Contre-exemples

1.3 Pseudo-code et complexité de Tree-Search-BF(T, k)

Tree-Search-BF(T, k)

```
1 if T == NULL then
2     return NULL
3
4 current = T.root
5 successor = NULL
6
7 while current  $\neq$  NULL do
8     cmp = k - diskFreeSpace(current.disk)
9     if cmp == 0 then
10         return current.disk
11     if cmp < 0 then
12         successor = current
13         current = current.left
14     else
15         current = current.right
16
17 if successor == NULL then
18     return NULL
19 return successor.disk
```

Nous pouvons à présent analyser la complexité dans le pire cas de notre code à travers le pseudo-code présenté ci-dessus. Cet algorithme commence par la racine de l'arbre et parcourt les sommets d'une branche sans jamais remonte, ainsi, l'algorithme parcourt au maximum la plus longue branche de l'arbre. Étant donné que nous avons implémenté un treap, la longueur des branches est bornée par n (où n est le nombre de sommets de l'arbre). Ainsi notre complexité finale est de l'ordre de $O(n)$. Remarquons qu'en pratique, le fait que la longueur d'une branche soit de longueur n est très improbable car le treap a pour but de limiter cela en augmentant fortement les probabilités que la longueur des branches soit logarithmique par rapport aux noeuds. Cependant, dans le pire cas si les priorités ne sont pas tirées de manière uniforme, on pourrait avoir du n .¹

1. <https://en.wikipedia.org/wiki/Treap>

1.4 Pseudo-code et complexité de Tree-Search-FF(T, size)

Tree-Search-FF(T, size)

```
1 if root == NULL then
2     return NULL
3
4 if root.subMax < size then
5     return NULL
6
7 currentBest = root
8
9 while currentBest.left ≠ NULL and currentBest.left.subMax ≥ size do
10     currentBest = currentBest.left
11
12 if diskFreeSpace(currentBest.disk) ≥ size then
13     return currentBest.disk
14 else
15     return Tree-Search-FF(currentBest.right, size)
```

Rappelons que nous travaillons sur un treap dont les clés sont l'ordre d'ouverture des disques. L'idée de cet algorithme est de parcourir l'arbre en descendant en partant de la racine, tout en voulant aller le plus à gauche, là où les disques sont les plus anciens. Donc nous partons de la racine, nous regardons si le sous arbre de gauche contient un disque qui peut accueillir un fichier de taille *size* (par l'intermédiaire de l'attribut *subMax*, qui contient la plus grande taille restante du disque de son sous arbre). S'il peut contenir un tel fichier, nous continuons à explorer dans cette direction. Au contraire, s'il ne peut pas, nous regardons si le disque actuel peut le contenir, dans ce cas, ce sera lui qui prendra le fichier. Dans l'autre cas, nous partons à droite et nous recommençons la procédure.

Nous pouvons à présent analyser la complexité dans le pire cas de notre code. Cet algorithme commence par la racine de l'arbre et parcourt les sommets d'une branche sans jamais remonter, ainsi, l'algorithme parcourt au maximum la plus longue branche de l'arbre. Étant donné que nous avons implémenté un treap, la longueur des branches est bornée dans le pire cas par n (où n est le nombre de sommets de l'arbre). Ainsi notre complexité finale est de l'ordre de $O(n)$.

1.5 Analyse de la complexité en temps dans le pire cas des quatre algorithmes

Notons que dans cette section, nous avons fait le choix d'utiliser n pour signifier le nombre de disques et N pour le nombre de fichiers. Nous avons fait la distinction entre les deux, bien que nous sommes dans le pire cas et que N pourrait convenir car le nombre de disques utilisé est borné par n .

NextFit

Nous commençons par trier la liste de fichiers par ordre décroissant selon leur taille, pour cela, nous utilisons un mergeSort qui a une complexité de l'ordre de $O(N \log(N))$. Ensuite, nous avons une boucle qui parcourt toute la liste de fichiers, et pour chaque fichier, nous faisons notre choix glouton. Comme nous choisissons simplement le disque courant s'il a de la place et sinon nous continuons en créant un nouveau, le choix glouton est d'un ordre constant. Cette boucle est donc de l'ordre de $\Theta(N)$. La complexité finale est $\Theta(N \log(N) + N)$, qui revient à $\Theta(N \log(N))$.

WorstFit

Comme pour le nextFit, nous faisons le tri qui est $O(N \log(N))$. Ensuite, comme pour le NextFit, nous avons une boucle qui parcourt la liste et insère à chaque itération le fichier dans le disque le moins rempli et cela en complexité $\Theta(\log(n))^2$, en effet, lorsque nous insérons, nous devons refaire notre tas correctement. Au final, nous avons $\Theta(N \log(N) + N \log(n))$, ce qui revient à $\Theta(N \log(N))$.

BestFit

Comme pour les deux algorithmes précédents, nous faisons le tri qui est $O(N \log(N))$. Ensuite, nous avons une boucle qui parcourt tous les fichiers comme dans les algorithmes précédents. Dans cette boucle, nous avons trois opérations qui sont $O(n)^2$. Ces trois opérations sont les appels de fonction suivants : `tree_search_bf`, `treap_delete`, `treap_insert`. Ainsi chaque itération de la boucle est de l'ordre de $O(3n)$ dans le pire cas. Au final notre complexité est de l'ordre de $O(N^2)$ dans le pire cas.

FirstFit

Comme pour les autres algorithmes précédents, nous faisons le tri qui est $O(N \log(N))$. Après, nous avons notre boucle principale qui parcourt les N fichiers. Ensuite, dans cette boucle, nous avons trois opérations qui ne sont de l'ordre constant : `tree_search_ff`, `treap_notify`, et `treap_insert` qui sont respectivement de l'ordre dans le pire cas $O(n)$. Si nous trouvons un disque adéquat, nous faisons le `treap_notify` au lieu de `treap_insert`, et à l'inverse nous faisons `treap_insert` au lieu de `treap_notify`. Nous avons donc au final dans la boucle $O(n) + \max(O(n), O(n))$, ce qui donne $O(2n)^2$. De plus, `treap_notify` pour re actualiser le champ `subMax` des sommets touchés, est bien $O(n)$ car si nous faisons l'hypothèse que le treap ne peut pas être équilibré (ce qui est très improbable), nous pourrions parcourir toute une branche qui serait de l'ordre de n . A noter que cela n'arrive jamais ou presque jamais en pratique car le treap permet qu'il y ait une très grande probabilité que la hauteur d'une branche soit logarithmique par rapport au nombre de noeuds. Pour le `treap_insert`, le même raisonnement s'applique. Pour finir, dans la boucle nous avons $O(n) + O(n)$, soit $O(n)$. Au final, la complexité totale du FirstFit est $\Theta(N^2)$.

2. Sans oublier que nous considérons le pire cas, nous avons n le nombre de disques ouverts qui est de l'ordre de $O(N)$.

2 Analyse empirique

2.1 Graphe sur l'évolution du gaspillage

Pour analyser l'évolution du gaspillage nous avons décidé d'expérimenter pour N de 10 000 à 100 000 par pas de 10 000.

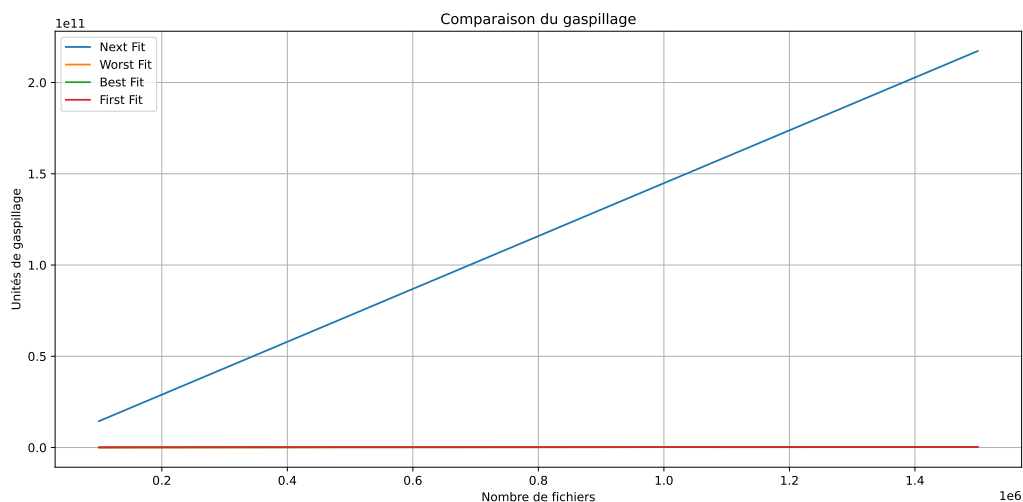


FIGURE 2 – Évolution du Gaspillage

Nous remarquons directement deux classes. D'abord le Next Fit qui semblent obtenir un gaspillage directement proportionnel au nombre de fichiers. En utilisant une régression linéaire, nous obtenons un coefficient d'environ 150 000 unités de gaspillage par fichier supplémentaire pour Next Fit. La deuxième classe, Best Fit, Worst Fit et First Fit, est par contre beaucoup plus efficace. Si l'on souhaite être efficace au niveau du gaspillage, nous avons intérêt à opter pour ces trois derniers algorithmes. Pour pouvoir mieux différencier les algorithmes de la deuxième classes nous devons utiliser un graphe qui compare uniquement ces trois algorithmes.

Voici un graphe qui compare Worst Fit, Best Fit et First Fit côte à côte.

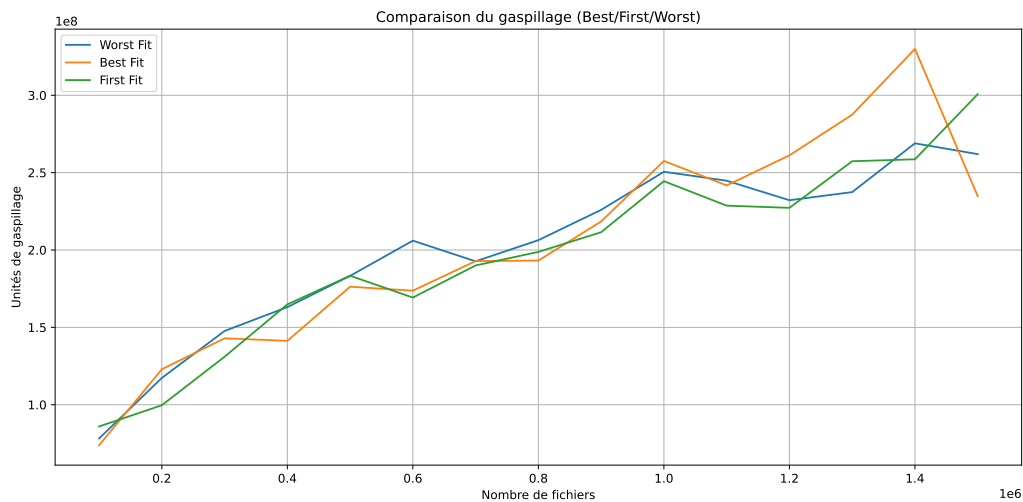


FIGURE 3 – Évolution du Gaspillage pour Worst Fit, Best Fit et First Fit

Premièrement, nous remarquons que le gaspillage semble être linéaire voir même inférieur pour les trois algorithmes. En utilisant une régression linéaire, nous obtenons un coefficient d'environ 115 unités de gaspillage par fichier supplémentaire pour Worst Fit, 141 Best Fit et 130 pour First Fit (plus de 1 000 fois moins que Next Fit). Deuxièmement, il est clair que parfois nos algorithmes ne sont pas strictement croissants au niveau du gaspillage. Ceci s'explique par le fait que nous testons nos algorithmes sur des fichiers générés aléatoirement. Au final, aucun des trois algorithmes ne se démarque. Comme les courbes sont proches au niveau du gaspillage, en réalité pour trouver un algorithme qui se démarque, il faudra comparer les différentes complexités temporelles.

2.2 Evolution des temps de calcul moyens en fonction de N

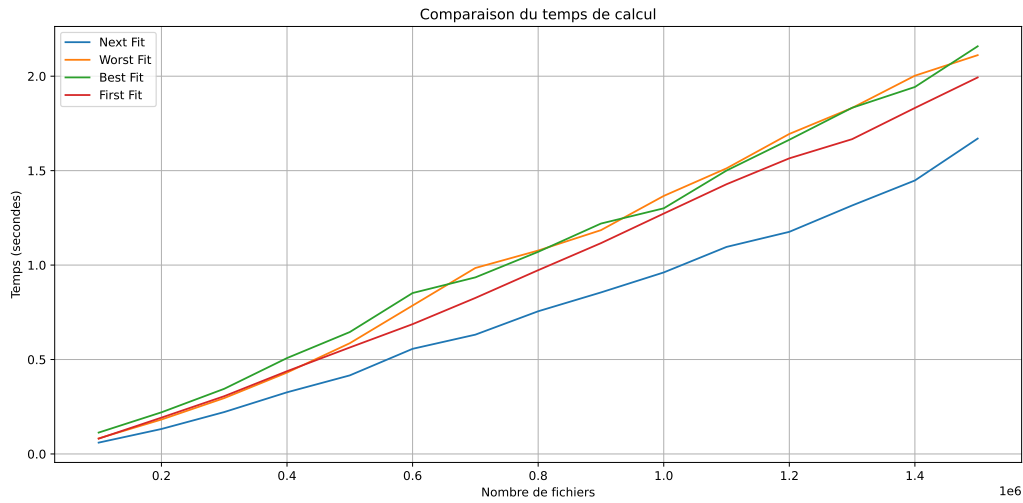


FIGURE 4 – Évolution du temps de calcul

Ce graphe peut surprendre au premier coup d'oeil car les courbes semblent linéarithmiques, ce qui est inattendu, au vu des pires cas pour Best Fit et First Fit. Cependant, le cas moyen semble plus intéressant. Tout d'abord nous pouvons rappeler que même si la complexité des algorithmes Treap est quadratique dans le pire cas, elle est linéarithmique dans le cas moyen, ce que nous observons ici. Avec une régression linéaire, nous obtenons un coefficient d'environ 10^{-6} secondes par fichier supplémentaire pour tous nos algorithmes, ce qui est quasi-négligeable. Par exemple, pour 1 million de fichiers supplémentaires, cela n'augmenterait que le temps d'exécution de 1 seconde. Next Fit reste le plus performant tout de même. Pour conclure, si nous devons utiliser un algorithme parmi ceux-ci, ce serait le Worst Fit, First Fit ou le Best Fit. En effet, ceux-ci sont extrêmement plus efficaces que le Next Fit au vu de l'analyse faite ci-dessus sans pour autant être beaucoup plus lent. Et de plus, nous obtenons cette efficacité pour une perte de temps négligeable. Au final, Worst Fit, First Fit et Best Fit sont meilleurs en pratique bien que dans le pire cas ils sont moins bons avec un treap. Notons que nous aurions pu obtenir une complexité de $\Theta(n \log(n))$ dans le pire cas en utilisant un AVL.